FormSubmission

Package: `@wlindabla/form_validator` — v3.0.0
Module: `FormSubmission`
Author: AGBOKOUDJO Franck — internationaleswebservices@gmail.com
GitHub: https://github.com/Agbokoudjo/form_validator

Table of Contents

1. Overview
2. Installation
3. Quick Start
4. Constructor
5. State Machine
6. Events
7. API Reference
8. Delegation
9. Advanced Usage
10. Integration Examples
11. Error Handling
12. FAQ

Overview

`FormSubmission` is a powerful module included in `@wlindabla/form_validator` that handles the **complete lifecycle of an HTML form submission** via the Fetch API.

It provides out of the box:

- A **state machine** to track every step of the submission process
- A **typed event system** that lets you hook into each lifecycle phase
- **Automatic server-side field error handling** (e.g. Symfony validation errors)
- A **confirmation gate** before any request is fired
- A **delegation system** for custom navigation/redirect logic
- Full **TypeScript support**

Installation

```bash
# npm
npm install @wlindabla/form_validator@3.0.0

# yarn
yarn add @wlindabla/form_validator@3.0.0

# pnpm
pnpm add @wlindabla/form_validator@3.0.0
```

Quick Start

```typescript
import { FormSubmission } from '@wlindabla/form_validator';

const form = document.querySelector<HTMLFormElement>('#my-form')!;

const submission = new FormSubmission(form, {
  url: '/api/contact',
  method: 'POST',
});

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  await submission.processStart();
});
```

Constructor

```typescript
new FormSubmission(
  form: HTMLFormElement,
  fetchRequestOptions: FetchRequestOptions,
  mustRedirect?: boolean,
  formEventDispatcher?: EventDispatcherInterface,
  delegateFormSubmission?: DelegateFormSubmissionInterface
)
```

Parameters

Parameter	Type	Required	Default
form	HTMLFormElement	✔	—
fetchRequestOptions	FetchRequestOptions	✔	—
mustRedirect	boolean	✖	false
formEventDispatcher	EventDispatcherInterface	✖	new BrowserEventDispatcher(window)
delegateFormSubmission	DelegateFormSubmissionInterface	✖	new DefaultNavigator()

Example

```typescript
```

```
import { FormSubmission } from '@wlindabla/form_validator';
import { BrowserEventDispatcher } from '@wlindabla/event_dispatcher';

const form = document.querySelector<HTMLFormElement>('#registration-form')!;

const submission = new FormSubmission(
  form,
  {
    url: '/api/register',
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'X-Requested-With': 'XMLHttpRequest'
    }
  },
  true, // redirect after success
  new BrowserEventDispatcher(window)
);
```

State Machine

`FormSubmission` tracks every step of the submission through the following states:

```
initialized ──► requesting ──► waiting ──► receiving ──► stopped
                              │
                              └──► error
```

States

State	Description
`initialized`	Default state — the instance has just been created
`requesting`	`processStart()` has been called and confirmed
`waiting`	The HTTP request has started
`receiving`	A successful response (2xx) has been received
`stopping`	`processStop()` has been called while a request was in progress
`stopped`	The process has fully ended (success or abort)
`error`	A network error or a server error response (4xx/5xx) occurred

Accessing the state

```typescript
const submission = new FormSubmission(form, options);

console.log(submission.state); // "initialized"

await submission.processStart();

console.log(submission.state); // "stopped" or "error"
```

Events

`FormSubmission` dispatches typed events at each stage of the lifecycle. You can listen to them on the `window` or `document` object.

Event Constants — `FormSubmitRequestEvents`

Constant	Value	Fired when
`FORM_SUBMIT_PREPARE_REQUEST`	`"form-prepare-request"`	Just before the request is sent — headers can be mutated here
`FORM_SUBMIT_START`	`"form-submit-start"`	The request has started
`FORM_SUBMIT_SUCCESS`	`"form-submit-success"`	A 2xx response was received
`FORM_SUBMIT_FAILED`	`"form-submit-failed"`	A 4xx or 5xx response was received
`FORM_SUBMIT_ERROR`	`"form-submit-error"`	A network/timeout/abort error occurred
`FORM_SUBMIT_END`	`"form-submit-end"`	The submission process has fully ended (always fired)

Event Classes

`PrepareRequestFormSubmitEvent`

Fired just before the request is sent. Use it to mutate headers.

```typescript
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_PREPARE_REQUEST, (e) => {
  const event = e as PrepareRequestFormSubmitEvent;
  console.log('Request about to be sent:', event.currentRequest);
  console.log('Form element:', event.formElement);
});
```

FormSubmitStartEvent

Fired when the request starts. The submitter button is disabled at this point.

```typescript
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_START, (e) => {
  const event = e as FormSubmitStartEvent;
  console.log('Submission started for form:', event.target);
  console.log('FormSubmission instance:', event.formSubmission);
});
```

FormSubmitSuccessEvent

Fired when a 2xx response is received.

```typescript
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_SUCCESS, (e) => {
  const event = e as FormSubmitSuccessEvent;
  const { fetchResponse } = event.resultHttpResponse;
  console.log('Success! Response data:', fetchResponse);
});
```

FormSubmitFailedEvent

Fired when a 4xx or 5xx response is received from the server.

```typescript
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_FAILED, (e) => {
  const event = e as FormSubmitFailedEvent;
  console.log('Server error:', event.response.statusCode);
  console.log('Failed request:', event.request);
  console.log('Form:', event.formElement);
});
```

FormSubmitRequestErrorEvent

Fired when a **network-level** error occurs (timeout, abort, no connection).

```typescript
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_ERROR, (e) => {
  const event = e as FormSubmitRequestErrorEvent;
  console.error('Network error:', event.requestError.message);
});
```

FormSubmitEndEvent

Always fired at the end of the process, regardless of success or failure.

```typescript
```

```
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_END, (e) => {
  const event = e as FormSubmitEndEvent;
  console.log('Submission finished. Result:', event.resultHttpResponse);
  // Re-enable UI, hide loaders, etc.
});
```

API Reference

processStart(): Promise<FetchResponseInterface | null>

Starts the form submission process. Awaits user confirmation before firing the request. Returns `null` if the user cancels or if the state is not `initialized`.

```typescript
const result = await submission.processStart();

if (result === null) {
  console.log('Submission was cancelled or already running.');
}
```

---

processStop(): void

Cancels the submission. If the request has not started yet (state is `initialized`), it transitions directly to `stopped`. Otherwise it calls `fetchRequest.cancel()`.

```typescript
```

```
// Cancel after 5 seconds
setTimeout(() => {
  submission.processStop();
}, 5000);
```

---

withHandleErrorsManyForm(value: boolean): void

Enables or disables automatic field-level error handling for server-side validation errors (e.g. Symfony 422 responses).

Enabled by default (`true`).

```typescript
// Disable automatic field error handling
submission.withHandleErrorsManyForm(false);
```

> See handleErrorsManyForm documentation for more details.

---

confirmMethodRequest ***(setter)***

Replaces the default browser `confirm()` dialog with a custom confirmation method (e.g. SweetAlert, a custom modal).

```typescript
```

```
// Example with SweetAlert2
submission.confirmMethodRequest = async (message, form, submitter) => {
  const result = await Swal.fire({
    title: 'Confirmation',
    text: message,
    icon: 'question',
    showCancelButton: true,
    confirmButtonText: 'Yes, send it!',
  });
  return result.isConfirmed;
};
```

---

prepareRequest(request: Request): void

Called internally just before the request is sent. Dispatches `PrepareRequestFormSubmitEvent`. Listen to this event to mutate request headers dynamically.

```typescript
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_PREPARE_REQUEST, (e) => {
  const event = e as PrepareRequestFormSubmitEvent;
  // Add a custom header dynamically
  event.currentRequest.headers.set('X-CSRF-Token', getCsrfToken());
});
```

Delegation

The `DelegateFormSubmissionInterface` allows you to implement custom logic at each lifecycle step without

modifying `FormSubmission` itself.

Interface

```typescript
interface DelegateFormSubmissionInterface {
  formSubmissionStarted(submission: FormSubmissionInterface): void;
  formSubmissionSucceededWithResponse(submission: FormSubmissionInterface, response: FetchResponseInterface
  formSubmissionFailedWithResponse(submission: FormSubmissionInterface, response: FetchResponseInterface): vo
  formSubmissionErrored(submission: FormSubmissionInterface, error: Error): void;
  formSubmissionFinished(submission: FormSubmissionInterface): void;
}
```

Custom Delegate Example

```typescript
```

```
import { DelegateFormSubmissionInterface, FormSubmissionInterface } from '@wlindabla/form_validator';
import { FetchResponseInterface } from '@wlindabla/http_client';

class MyAppNavigator implements DelegateFormSubmissionInterface {
  formSubmissionStarted(submission: FormSubmissionInterface): void {
    console.log('Form submission started');
    document.querySelector('#loader')?.classList.remove('hidden');
  }

  formSubmissionSucceededWithResponse(
    submission: FormSubmissionInterface,
    response: FetchResponseInterface
  ): void {
    document.querySelector('#loader')?.classList.add('hidden');
    window.location.href = '/dashboard'; // Custom redirect
  }

  formSubmissionFailedWithResponse(
    submission: FormSubmissionInterface,
    response: FetchResponseInterface
  ): void {
    document.querySelector('#loader')?.classList.add('hidden');
    alert(`Error ${response.statusCode}: please check your inputs.`);
  }

  formSubmissionErrored(submission: FormSubmissionInterface, error: Error): void {
    document.querySelector('#loader')?.classList.add('hidden');
    alert('Network error. Please check your connection.');
  }

  formSubmissionFinished(submission: FormSubmissionInterface): void {
    console.log('Submission process fully ended. Final state:', submission.state);
```

```
    }
}

// Use it
const submission = new FormSubmission(
    form,
    { url: '/api/contact', method: 'POST' },
    false,
    undefined,
    new MyAppNavigator()
);
```

Advanced Usage

Confirmation attribute on the submitter button

You can configure the confirmation message directly in HTML using the `data-iwas-confirm` attribute on the submit button:

```html
<form id="delete-form" method="POST" action="/api/user/delete">
    <button
        type="submit"
        data-iwas-confirm="Are you sure you want to delete your account? This action is irreversible."
    >
        Delete my account
    </button>
</form>
```

```typescript
const form = document.querySelector<HTMLFormElement>('#delete-form')!;
const submission = new FormSubmission(form, { url: '/api/user/delete', method: 'POST' });

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  await submission.processStart();
});
```

Disabling automatic field error handling

By default, when the server returns a `422 Unprocessable Entity`, `FormSubmission` automatically maps field errors to their corresponding form inputs via `handleErrorsManyForm`. To disable this behavior:

```typescript
const submission = new FormSubmission(form, options);
submission.withHandleErrorsManyForm(false);

// Now you handle field errors manually via the FORM_SUBMIT_FAILED event
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_FAILED, (e) => {
  const event = e as FormSubmitFailedEvent;
  if (event.response.statusCode === 422) {
    const errors = event.response.data;
    // your custom field error rendering logic
  }
});
```

Aborting a long-running request

```typescript
const submission = new FormSubmission(form, {
  url: '/api/heavy-process',
  method: 'POST',
});

const cancelBtn = document.querySelector<HTMLButtonElement>('#cancel-btn')!;

cancelBtn.addEventListener('click', () => {
  submission.processStop();
  console.log('Submission cancelled. State:', submission.state); // "stopping" or "stopped"
});

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  await submission.processStart();
});
```

Integration Examples

With Symfony (422 validation errors)

Symfony returns validation errors as a JSON object keyed by field name. `FormSubmission` handles this automatically when the response status is `422`:

```typescript
```

```
// Symfony returns:
// { "email": "This value is not a valid email.", "username": "This value is too short." }

const submission = new FormSubmission(form, {
  url: '/api/register',
  method: 'POST',
  headers: { 'Accept': 'application/json' }
});

// withHandleErrorsManyForm is true by default:
// errors are automatically displayed next to the correct fields
await submission.processStart();
```

With SweetAlert2

```typescript
```

```
import Swal from 'sweetalert2';

const submission = new FormSubmission(form, { url: '/api/order', method: 'POST' });

// Replace native confirm() with SweetAlert2
submission.confirmMethodRequest = async (message) => {
  const result = await Swal.fire({
    title: 'Confirm your order',
    text: message,
    icon: 'warning',
    showCancelButton: true,
    confirmButtonColor: '#3085d6',
    cancelButtonColor: '#d33',
    confirmButtonText: 'Yes, place order!'
  });
  return result.isConfirmed;
};

// Show a success toast on completion
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_SUCCESS, () => {
  Swal.fire('Done!', 'Your order has been placed.', 'success');
});

// Show an error alert on network failure
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_ERROR, (e) => {
  const event = e as FormSubmitRequestErrorEvent;
  Swal.fire('Network Error', event.requestError.message, 'error');
});
```

With Vue 3

```typescript
```

```typescript
// composables/useFormSubmission.ts
import { ref, onUnmounted } from 'vue';
import {
  FormSubmission,
  FormSubmitRequestEvents,
  FormSubmitEndEvent
} from '@wlindabla/form_validator';

export function useFormSubmission(form: HTMLFormElement, url: string) {
  const isLoading = ref(false);
  const error = ref<string | null>(null);

  const submission = new FormSubmission(form, { url, method: 'POST' });

  const onStart = () => { isLoading.value = true; error.value = null; };
  const onEnd = (e: Event) => {
    isLoading.value = false;
    const event = e as FormSubmitEndEvent;
    if (!event.resultHttpResponse.success) {
      error.value = 'Submission failed. Please try again.';
    }
  };

  window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_START, onStart);
  window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_END, onEnd);

  onUnmounted(() => {
    window.removeEventListener(FormSubmitRequestEvents.FORM_SUBMIT_START, onStart);
    window.removeEventListener(FormSubmitRequestEvents.FORM_SUBMIT_END, onEnd);
    submission.processStop();
  });
```

```
    return { submission, isLoading, error };
}
```

With React

```typescript
```

```typescript
// hooks/useFormSubmission.ts
import { useState, useEffect, useRef } from 'react';
import {
  FormSubmission,
  FormSubmitRequestEvents,
} from '@wlindabla/form_validator';

export function useFormSubmission(formRef: React.RefObject<HTMLFormElement>, url: string) {
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const submissionRef = useRef<FormSubmission | null>(null);

  useEffect(() => {
    if (!formRef.current) return;

    submissionRef.current = new FormSubmission(formRef.current, {
      url,
      method: 'POST',
    });

    const handleStart = () => { setIsLoading(true); setError(null); };
    const handleEnd = () => setIsLoading(false);
    const handleError = () => setError('A network error occurred.');

    window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_START, handleStart);
    window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_END, handleEnd);
    window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_ERROR, handleError);

    return () => {
      window.removeEventListener(FormSubmitRequestEvents.FORM_SUBMIT_START, handleStart);
      window.removeEventListener(FormSubmitRequestEvents.FORM_SUBMIT_END, handleEnd);
      window.removeEventListener(FormSubmitRequestEvents.FORM_SUBMIT_ERROR, handleError);
```

```
        submissionRef.current?.processStop();
      };
    }, [formRef, url]);

    const submit = async (e: React.FormEvent) => {
      e.preventDefault();
      await submissionRef.current?.processStart();
    };

    return { submit, isLoading, error };
  }
```

Error Handling

Error types and their corresponding events

Error type	Event fired	Class
Network failure (no connection)	FORM_SUBMIT_ERROR	FormSubmitRequestErrorEvent
Request timeout	FORM_SUBMIT_ERROR	FormSubmitRequestErrorEvent
Request aborted	FORM_SUBMIT_ERROR	FormSubmitRequestErrorEvent
Server error 4xx / 5xx	FORM_SUBMIT_FAILED	FormSubmitFailedEvent
Validation error 422 (auto-handled)	FORM_SUBMIT_FAILED	FormSubmitFailedEvent

Complete error handling example

```typescript
// Network / timeout / abort errors
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_ERROR, (e) => {
  const event = e as FormSubmitRequestErrorEvent;
  console.error('[Network Error]', event.requestError.message);
  showToast('Connection failed. Please check your internet.', 'error');
});

// Server-side errors (4xx / 5xx)
window.addEventListener(FormSubmitRequestEvents.FORM_SUBMIT_FAILED, (e) => {
  const event = e as FormSubmitFailedEvent;
  const { statusCode, data } = event.response;

  if (statusCode === 401) {
    window.location.href = '/login';
  } else if (statusCode === 403) {
    showToast('You are not authorized to perform this action.', 'warning');
  } else if (statusCode === 422) {
    // Field errors are already auto-handled by handleErrorsManyForm
    // unless you called withHandleErrorsManyForm(false)
    console.warn('Validation errors:', data);
  } else {
    showToast(`Server error (${statusCode}). Please try again later.`, 'error');
  }
});
```

FAQ

Q: Can I use `FormSubmission` with a `GET` form?
Yes. `FormData` is only attached automatically for `POST`, `PUT`, and `PATCH` methods. For `GET` requests, query parameters should be set via `fetchRequestOptions.url`.

Q: What happens if the user clicks submit twice?
The state machine prevents double submission. Once the state moves from `initialized` to `requesting`, any further calls to `processStart()` return `null` immediately.

Q: How do I add custom request headers?
Listen to the `FORM_SUBMIT_PREPARE_REQUEST` event and mutate `event.currentRequest.headers`, or pass them directly in `fetchRequestOptions.headers`.

Q: Can I use a custom HTTP client?
`FormSubmission` uses `FetchRequest` from `@wlindabla/http_client` internally. To customize behavior, use `fetchRequestOptions` or listen to lifecycle events.

Q: Is `FormSubmission` compatible with TypeScript strict mode?
Yes. All classes, events, and interfaces are fully typed and compatible with `"strict": true` in `tsconfig.json`.

Q: Where can I find the `handleErrorsManyForm` documentation?
See the dedicated documentation at:
https://github.com/Agbokoudjo/form_validator/docs/_Utils/form.md